

- 在工业场景中，工控程序在异常情况下需要调整 LED 灯的显示颜色和闪烁频率。

分析以上案例，我们发现 read/write 接口并不足以覆盖设备的所有操作需求，尤其是在控制设备的工作模式方面。为此，类 UNIX 操作系统给设备文件提供了一个足够通用的操作接口，即 I/O Control 接口——ioctl。如代码片段 13.12 所示，ioctl 允许应用程序和设备驱动之间自定义一组操作请求：应用程序在操作设备时，需要同时发送请求码和相应参数，设备驱动则根据请求码的语义调用对应的 I/O 处理逻辑。Windows 操作系统上用于 I/O 控制的接口称为 DeviceIoControl，其作用和 UNIX 系统上的 ioctl 相同。

代码片段 13.12 用于应用程序操作设备的通用接口：ioctl

```
1 #include <sys/ioctl.h>
2
3 int ioctl(int fd, unsigned long request, ...);
```

代码片段 13.13 展示了如何在 UNIX 系统上使用 ioctl 接口周期性地控制 LED 设备的开和关两个动作。应用程序首先将设备文件系统下的 LED 设备文件打开，然后使用 ioctl 对文件描述符进行操作，操作内容为开关动作对应的操作码。操作码是应用程序和设备驱动事先商量好的“魔数”。由于魔数被直接硬编码在应用程序中，该做法在一定程度上损害了应用程序的可移植性。

代码片段 13.13 使用 ioctl 操作 LED 设备

```
1 #define LED_ALL_ON _IO('L', 0x1234)
2 #define LED_ALL_OFF _IO('L', 0x5678)
3
4 int main(void)
5 {
6     int fd = open("/dev/led", O_RDWR);
7     if (fd < 0) exit(1);
8
9     while(1) {
10         ioctl(fd, LED_ALL_ON);
11         sleep(1);
12         ioctl(fd, LED_ALL_OFF);
13         sleep(1);
14     }
15
16     close(fd);
17     return 0;
18 }
```

如代码片段 13.14 所示，LED 驱动程序在收到由应用程序发来的操作码后，会根据不同的操作码内容对 LED 设备进行相应的 MMIO 操作，从而实现 LED 的全亮和全灭。和 read/write 接口一样，设备驱动也需要向操作系统注册 ioctl 接口，这样操作系

统才能正确地调用该接口。

代码片段 13.14 用户程序和设备驱动约定好相同的操作码，设备驱动根据不同请求的操作码执行对应操作

```
1 #define LED_ALL_ON _IO('L', 0x1234)
2 #define LED_ALL_OFF _IO('L', 0x5678)
3
4 long led_ioctl(..., unsigned int cmd, ...)
5 {
6     switch (cmd) {
7         case LED_ALL_ON:
8             *gpio_data |= 0x0; break;
9         case LED_ALL_OFF:
10            *gpio_data &= ~0x0; break;
11         default: break;
12     }
13     ...
14 }
15
16 static struct file_operations fops = {
17     .ioctl = led_ioctl,
18 };
```

13.4 应用 I/O 框架

本节主要知识点

- 为什么要为应用程序提供应用层的 I/O 库？
- 用户态 I/O 管理和操作系统 I/O 管理有什么优点与缺点？

13.4.1 应用层 I/O 库

有时候操作系统没法提供足够适合应用程序特点的 I/O 策略。为了支持定制化的 I/O 策略，操作系统引入了应用层 I/O 库。I/O 库作为应用程序的一部分，通常以共享库的形式直接与应用程序链接，旨在提供更高的性能和更为灵活的设备管理能力。

libc

学过 C 语言的读者应该记得 C 程序使用了 `fopen`、`fclose`、`fread`、`fwrite` 等一系列以 `f` 开头的 I/O 库函数，这组库函数不同于文件系统的系统调用（`open`、`close`、`read`、`write`）。库函数属于高层接口（High-level API），而系统调用接口属于低层接口（Low-level API）。前者返回文件指针，而后者返回内核提供的文件描述符。

请思考代码片段 13.15 和代码片段 13.16 中，哪一段代码的运行效率更高呢？

代码片段 13.15 使用 fwrite 接口不断追加写文件 100 万次

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <string.h>
4
5 int main(void)
6 {
7     const char * line = "Hello World!\n";
8
9     FILE * fp = fopen("file.txt", "a");
10    for (int i = 0; i < 1000000; i++) {
11        fwrite(line, strlen(line), 1, fp);
12    }
13    fclose(fp);
14    return 0;
15 }
```

代码片段 13.16 使用 write 接口不断追加写文件 100 万次

```
1 #include <fcntl.h>
2 #include <unistd.h>
3 #include <string.h>
4
5 int main(void)
6 {
7     const char * line = "Hello World!\n";
8
9     int fd = open("file.txt", O_CREAT | O_WRONLY | O_APPEND, 0600);
10    for (int i = 0; i < 1000000; i++) {
11        write(fd, line, strlen(line));
12    }
13    close(fd);
14    return 0;
15 }
```

代码片段 13.15 和代码片段 13.16 分别使用 C 库函数的 I/O 库接口和 POSIX 系统调用标准文件 I/O 接口对文件进行追加写入 100 万行的操作。C 库函数的 I/O 库接口属于自带缓冲区管理的 I/O 函数（Buffered I/O）。基于缓冲区的应用层 I/O 实现可以有效避免频繁的系统调用带来的上下文切换，提供更好的性能支持。其性能优化的本质是借助缓冲区实现 I/O 读写操作的批处理。

练习

- 13.1 请在代码片段 13.15 和代码片段 13.16 的合适位置插入高精度时间获取函数（使用 C 库的 `gettimeofday`，或者 C++ 库的 `chrono`）计算两者的性能差距。

liburing 与 io_uring

如果应用和设备之间需要频繁地交换数据（如应用程序希望从磁盘的不同位置不断读取文件信息），那么光靠 libc 的应用层缓冲区就不足以满足低开销的需求了——libc 的缓冲区更适合来自同一侧的读写访问的批处理。同样可以引入缓冲区来进行优化，但这个缓冲区可以在应用程序和操作系统之间共享，从而减少数据的来回拷贝（如宏内核的用户态和内核态之间、微内核的不同进程之间）。

由于现代高性能 I/O 设备的出现（如 SSD 存储设备），Linux 内核从 5.1 版本开始提供名为 io_uring 的优化支持^[10]。如图 13.21 所示，io_uring 使用两个环形队列来实现 I/O 请求的收发：提交队列（Submission Queue）用于应用程序向内核发送系统调用请求，而完成队列（Completion Queue）用于收集系统调用返回的结果。提交队列和完成队列被内核和应用同时共享。

在多核 CPU 架构下，内核线程和应用线程可以运行在不同的 CPU 核心上。双方线程通过轮询共享的提交队列和完成队列的方式完成系统调用的请求和返回，而不必发生传统系统调用时的上下文切换（请回顾 3.2.6 节）。为了提高处理性能，双方线程可以进一步采取批处理的方式提高 I/O 吞吐。

为了屏蔽 io_uring 的底层实现细节，liburing 作为应用层 I/O 库，为应用程序提供了更为友好、简洁的高层开发接口^[11]。

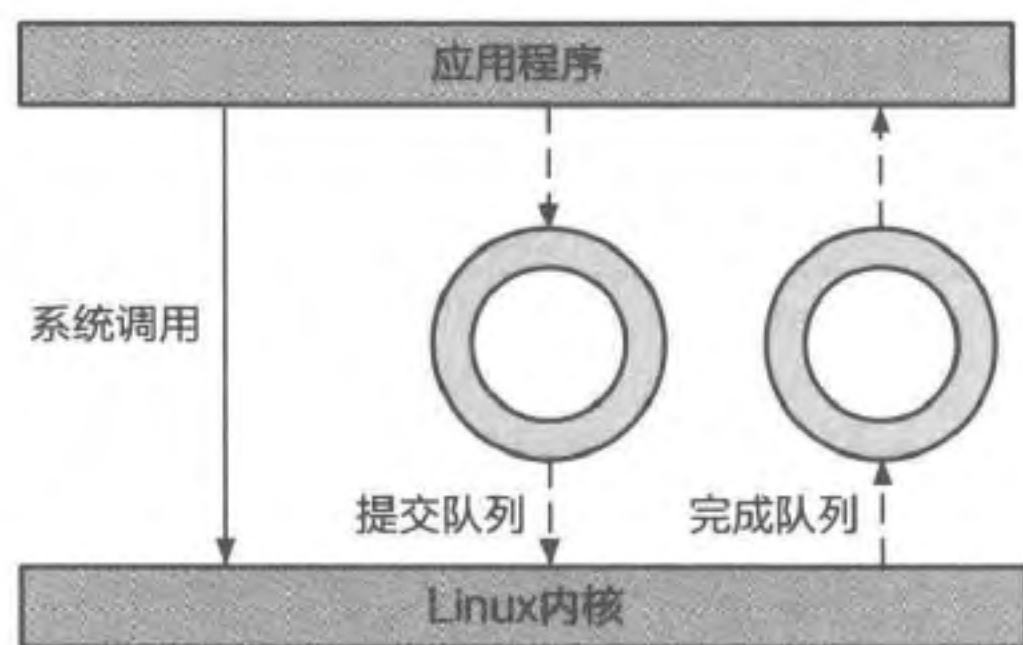


图 13.21 io_uring 基于提交队列和完成队列实现 I/O 系统调用的高性能处理。实线表示传统的系统调用（需发生上下文切换），虚线则表示应用进程和内核线程通过共享的双环形缓冲区完成的 I/O 操作

13.4.2 用户态 I/O

无论是基于系统调用还是 I/O 库接口，应用程序默认操作的设备对象都是操作系统提供的逻辑设备。操作系统在应用与设备之间，替应用操作实际的设备——这种间接的方式会引入性能的开销。请思考如下场景：数据中心通常会配置防火墙服务以防遭到网络攻击。防火墙软件需要不断监视网卡的网络包流量，并对可疑的网络包进行拦截。防火墙软件的处理性能决定了数据中心服务对外的响应延迟。假设工程师已部署了高性能网卡，并且启用了 Linux 的 NAPI，但性能始终不够理想。经过分析发现，主要原因正是网络数据在网卡驱动（内核态）和防火墙软件（用户态）之间大量的来回拷贝。

是否可以让应用程序直接而不是间接地操作物理设备呢？可以。一种直观的思路是允许防火墙软件直接操作网卡的 DMA 缓冲区。为了实现这一目标，Intel 联合其他网卡制造商共同开发了一套高性能的用户态网络 I/O 框架——数据平面开发套件（Data Plane Development Kit, DPDK）^[12]。DPDK 在设计上采取旁路内核（Kernel-bypass）的设计，

即网络包的收发处理基本不需要 Linux 内核的参与。DPDK 的设计思路如下。

用户空间驱动

为了能在用户态同网卡设备进行交互，DPDK 需要在用户态直接执行网卡驱动代码。其做法是将设备寄存器直接映射到应用自身的进程地址空间中，进而让 DPDK 的用户态驱动通过 MMIO 操作设备。

正如操作系统为应用程序的开发提供统一设备文件系统和 I/O 使用接口一样，Linux 提供了用户态驱动开发框架，即 UIO (Userspace I/O)。Linux 将 UIO 设备抽象为路径为 `/dev/uio [ X ]` 的设备文件。应用程序通过打开 UIO 设备文件获取设备的 I/O 空间和中断信息，同时自行决定如何操作和响应设备。

为了创建设备文件，UIO 驱动编写者需要在 Linux 系统中注册一个简单的 UIO 内核模块。代码片段 13.17 展示了 UIO 内核模块需要注册的结构体信息，其中，`mmap` 用于将设备的地址空间和端口空间映射到用户地址空间，`irqcontrol` 用于开启或屏蔽中断。UIO 驱动框架提供默认的读语义，`read` 会将用户空间驱动阻塞在等待中断上，当中断到来时 `read` 会立即返回，在返回值中表明一共发生了几次中断，UIO 用户驱动通常不使用 `write` 接口，而是从已经经过 `mmap` 处理的内存区间直接与设备交换数据。

代码片段 13.17 Linux 中 `uio_info` 结构体的定义

```

1 //include/linux/uio_driver.h
2
3 struct uio_info {
4     struct uio_device * uio_dev;
5     const char * name;           // UIO 设备名称
6     const char * version;       // UIO 设备版本号
7
8     // 用于映射到用户空间的内存和端口
9     struct uio_mem mem[MAX_UIO_MAPS];
10    struct uio_port port[MAX_UIO_PORT_REGIONS];
11
12    // 中断号
13    long irq;
14    // 中断处理函数，通常只负责确认中断，而不做数据处理
15    irqreturn_t (* handler) (int irq, struct uio_info *dev_info);
16
17    // 和用户空间驱动代码交互的接口
18    int (* mmap) (struct uio_info *info, struct vm_area_struct *vma);
19    int (* open) (struct uio_info *info, struct inode *inode);
20    int (* release) (struct uio_info *info, struct inode *inode);
21    int (* irqcontrol) (struct uio_info *info, s32 irq_on);
22 };

```

轮询模式驱动

内核态驱动的中断响应可以通过硬件机制实现，但用户态驱动的中断响应就没有那么直接了。中断发生时，CPU 首先跳转到 UIO 内核模块提前注册的中断处理函数处，该

函数通知操作系统的调度器，在中断处理函数返回后通过调度器进一步唤醒 UIO 用户态驱动正在等待中断的线程（该线程被阻塞而默认处于睡眠状态，直到中断发生）。整个过程涉及的上下文切换次数更多、时延更长，并不能达到预想中高性能处理网络包的效果。

为了避免中断带来的开销，DPDK 使用基于轮询模式的驱动程序（Poll Mode Drivers, PMD）。轮询模式驱动的处理思路和 NAPI 十分相似。如图 13.22 所示，基于 PMD 的网卡驱动首先通过 UIO 框架将设备的寄存器空间和 DMA 描述符映射到用户空间，随后开启中断。当网卡产生中断时，CPU 跳转到 PMD 网卡驱动注册的 UIO 内核态中断处理函数，该函数随即唤醒 PMD 被阻塞的处理线程，并通知网卡中断事件的到来。PMD 的处理线程被唤醒后，将立即屏蔽网卡中断，然后以轮询的方式在用户态操作网卡描述符并读写对应的 DMA 缓冲区，实现低延迟的网络数据包处理。不再有数据包到来后，PMD 线程停止收包，并再次将中断模式打开。

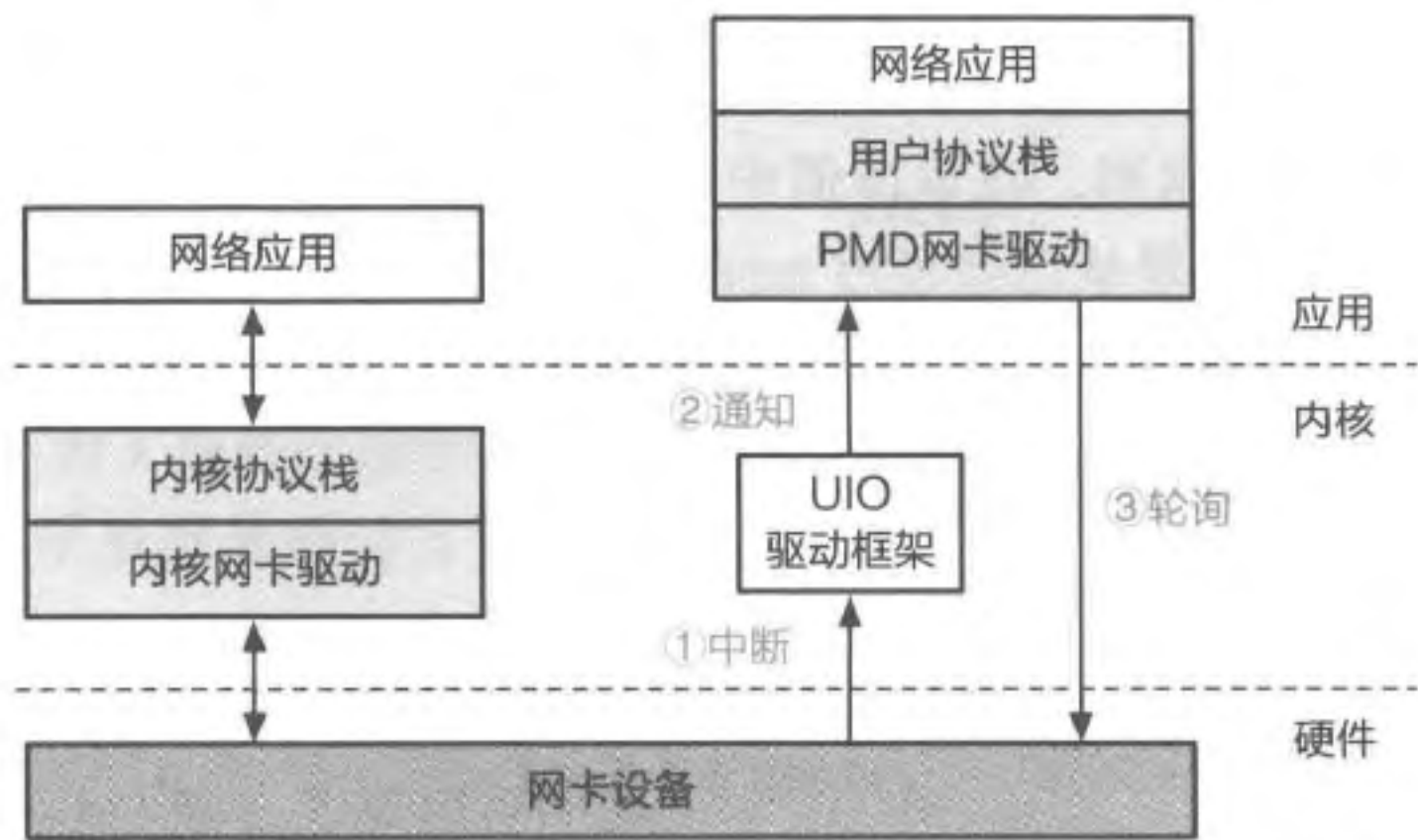


图 13.22 传统 Linux 内核网络 I/O 与基于 UIO 框架的 DPDK 网络层次示意图

13.5 案例分析：Android 操作系统的硬件抽象层

Android 操作系统是由谷歌公司和开放手机联盟共同主导开发的基于 Linux 内核的操作系统，主要面向移动端智能设备。Android 操作系统在 Linux 内核之上增加了一层硬件抽象层（Hardware Abstract Layer, HAL）。Android 引入硬件抽象层的主要原因可以归结成以下两个方面：

- Linux 宏内核属性使得设备驱动通常运行在内核态，内核驱动开发时不可避免地依赖内核驱动框架和各种子系统的支持，导致驱动实现与特定内核版本产生强耦合，阻碍了 Android 驱动生态的独立演进与升级。
- Linux 内核采用 GPLv2 开源协议，该协议具有“传染性”，这意味着运行在内核空间的设备驱动也必须一并开放源码。驱动开源会导致硬件设备的实现细节也被公开，一些设备厂商因此担心其硬件竞争力受到影响。